

Developing a terminal UI in Go with Bubble Tea

Developing CLIs and TUIs in Go is fun, and there are really good packages out there to make it so. Let's review one of them - Bubble Tea.



ALEX PLIUTAU

JUL 07, 2025



16



2



1

Share

Many developers love using command line tools for daily development tasks, at least I do. It's fun, they are usually performant. For example there are many Desktop applications to work with git, though I believe that the majority of programmers love to use git CLI (or TUIs like [lazygit](#)) as it's faster and can be automated, and is the same on all environments: be it your dev machine or server. There are 2 main flavours of CLIs: one is imperative, command-based CLIs, such as git again, or kubectl, where you define everything in prompt using sub-commands and flags. And the second is so-called Terminal Apps or Terminal UI (commonly called TUI) which is more interactive, and is basically a whole application that runs in your terminal.

For example, there is a [circumflex](#) TUI that lets you read Hacker News from your Terminal.

clx

clx best • ask • show

1. [Honda conducts successful launch and landing of experimental reusable rocket](#) (global.honda)
583 points by LorenDB 5 hours ago | 203 comments
2. [Resurrecting a dead torrent tracker and finding 3M peers](#) (kianbradley.com)
177 points by k-ian 3 hours ago | 59 comments
3. [Building Effective AI Agents](#) (anthropic.com)
83 points by Anon84 3 hours ago | 12 comments
4. [Making 2.5 Flash and 2.5 Pro GA, and introducing Gemini 2.5 Flash-Lite](#) (blog.google)
210 points by meetpateltech 4 hours ago | 125 comments
5. [AMD's CDNA 4 Architecture Announcement](#) (chipsonline.com)
50 points by rbanffy 3 hours ago | 9 comments
6. [Programming Language Design in the Era of LLMs](#)
23 points by gopiandcode an hour ago | 7 comments
7. [Real-time action chunking with large models](#)
15 points by pr337h4m an hour ago
8. [Time Series Forecasting with Graph Transformers](#)
33 points by turntable_pride 2 hours ago | 12 comments

And there are more examples like these lists just few.

Developing CLIs and TUIs in Go is not easy, so there are packages out there to make it so. For example, [command-based imperative CLIs](#) you can use [Francia](#), which was used to build popular CLIs.

And when it comes to terminal apps, you can use [Bubble Tea](#) to build beautiful interactive TUIs.



Discover more from [packagemain.tech](#)

Welcome to [packagemain.tech](#), your one-stop shop for mastering Backend, Cloud, Kubernetes, Microservices, APIs, and more. We'll provide you with weekly curated content.
Over 11,000 subscribers

Subscribe

By subscribing, I agree to Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).

Already have an account? [Sign in](#)



For example the [circumflex](#) TUI that we've seen before was developed using Bubble Tea.

Cut Code Review Time & Bugs in Half (Sponsor)

**Cut Code Review Time
and Bugs in *Half*
with  CodeRabbit**

```
src/App/App.jsx
...   ...   @@ -247,6 +247,7 @@ function App({ dexieCurrentUserInstance }) {
247   247   like so: https://preview.app2.datamermid.org/123/index.html */
248   248   <Route exact path="/index.html" element={<Navigate to="/projects"
249   249   <Route path="/" element={<PageNotFound /> } /> } />
250   250   <Route path="/" element={<PageNotFound /> } /> } />

coderabbitai bot yesterday
Verification agent
Analysis chain
Duplicate Catch-All Route Detected

The file src/App/App.jsx currently defines two catch-all routes that render the <PageNotFound /> component:
• Line 249: <Route path="/" element={<PageNotFound /> } />
• Line 250: <Route path="/" element={<PageNotFound /> } />

Both routes serve the same purpose of catching undefined paths. To clean up the code and avoid potential confusion, please remove one of these redundant route definitions. For consistency, it's recommended to keep the <Route path="/" element={<PageNotFound /> } /> and remove the other.

Refactor suggestion:

<Route path="/" element={<PageNotFound /> } />
- <Route path="/" element={<PageNotFound /> } />

Committable suggestion
```

Code reviews are critical but time-consuming. [CodeRabbit](#) acts as your AI co-pilot, providing instant Code review comments and

potential impacts of every pull request.

Beyond just flagging issues, CodeRabbit provides one-click fix suggestions and lets you define custom code quality rules using AST Grep patterns, catching subtle issues that traditional static analysis tools might miss.

CodeRabbit has so far reviewed more than 10 million PRs, installed on 1 million repositories, and used by 70 thousand Open-source projects. CodeRabbit is free for all open-source repo's!

Note-taking TUI

In this article we will build a terminal-based note taking app using Bubble Tea and some other libraries. It will be a very simple application with SQLite store for storing the notes.

Installation

If you don't have Bubble Tea installed yet, run the following commands to install it as well as few other adjacent packages :

```
go get github.com/charmbracelet/bubbletea
go get github.com/charmbracelet/lipgloss
go get github.com/charmbracelet/bubbles
```

Bubble Tea is usually used with other libraries, as you can see above, we installed them too.

[lipgloss](#) is a great styling library from Charm.

And `bubbles` is a TUI components library for Bubble Tea (also from Charm). For example File picker:

Bubble Tea Architecture

Before jumping to our program, let's review how Bubble Tea actually works.

Bubble Tea is based on the functional design paradigms of [The Elm Architecture](#), which happens to work nicely with Go. It's a delightful way to build applications.

Bubble Tea programs are comprised of a model that describes the application state and three simple methods on that model:

- **Init**, a function that returns an initial command for the application to run.
- **Update**, a function that handles incoming events and updates the model accordingly.
- **View**, a function that renders the UI based on the data in the model.

Now, how does it translate to code?

Model

A good place to start is the Model. Our note-taking application can be in multiple states (list view, edit title view, edit body view), and as you can imagine we will need UI elements such as text input, textarea, list. What's cool is that our main model can contain other models as well, so in a way we're building a tree of models.

So here is how our model may look like:

[model.go](#)

```
package main

import (
    "log"

    "github.com/charmbracelet/bubbles/textarea"
    "github.com/charmbracelet/bubbles/textinput"
    tea "github.com/charmbracelet/bubbletea"
)

const (
    listView uint = iota
    titleView
    bodyView
)

type model struct {
    store      *Store
```



```

    state      uint
    textarea   textarea.Model
    textinput  textinput.Model
    currNote   Note
    notes      []Note
    listIndex  int
}

func NewModel(store *Store) model {
    notes, err := store.GetNotes()
    if err != nil {
        log.Fatalf("unable to get notes: %v", err)
    }

    return model{
        store:      store,
        state:      listView,
        textarea:   textarea.New(),
        textinput:  textinput.New(),
        notes:      notes,
    }
}

func (m model) Init() tea.Cmd {
    return nil
}

```

As you can see our model is basically a state of our application. It also knows about the notes storage (sqlite in my case, but we will not focus on much on that). **NewModel()** function creates a new fresh state, and **Init()** is empty in our case, as initial command for the application is not required and not needed in our case.

Main Loop

With the model in place we can initiate a Bubble Tea program in [main.go](#)

```

package main

import (
    "log"

    tea "github.com/charmbracelet/bubbletea"
)

```

```

func main() {
    store := new(Store)
    if err := store.Init(); err != nil {
        log.Fatalf("unable to init store: %v", err)
    }

    m := NewModel(store)

    p := tea.NewProgram(m)
    if _, err := p.Run(); err != nil {
        log.Fatalf("unable to run tui: %v", err)
    }
}

```

As you can see, we can pass our model to **tea.NewProgram()** and Bubble Tea will do the rest for us, assuming that our Model implements the interface with **Init()**, **Update()**, **View()** methods.

```

type Model interface {
    Init() Cmd
    Update(msg Msg) (Model, Cmd)
    View() string
}

```

Update

The **Update()** method handles user input (or any other events such as Ticks for example).

Here we can react to various events and update our model accordingly. For example, when a user presses the hotkeys, we switch to another view.

What's interesting is that our Model contains other models so we must propagate the **Update()** accordingly.

```

func (m model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
    m.textarea, _ = m.textarea.Update(msg)
    m.textinput, _ = m.textinput.Update(msg)

    switch msg := msg.(type) {

```

```

// handle key strokes
case tea.KeyMsg:
    key := msg.String()

    switch m.state {
    // List View key bindings
    case listView:
        switch key {
        case "q":
            return m, tea.Quit
        case "n":
            // ...
        case "up", "k":
            // ...
        case "down", "j":
            // ...
        case "enter":
            // ...
        }

    // Title Input View key bindings
    case titleView:
        switch key {
        case "enter":
            // ...
        case "esc":
            m.state = listView
        }

    // Body Textarea key bindings
    case bodyView:
        switch key {
        case "ctrl+s":
            // ...
        case "esc":
            // ...
        }
    }

    return m, nil
}

```

As you can see here in our **Update()** function we react to the following keystrokes:

- q - quit the app

- n - new note
- j,k - move up,down between notes
- enter - open note
- ctrl+s - save note
- esc - exit the step

The **Msg** type in Bubble Tea is flexible and can carry various data. In this scenario, it resembles browser events in JavaScript. For instance, a timer event might not carry data, while a click event specifies what was clicked.

But be aware, that messages are not necessarily received in the order they are sent, in Go, if you have more than one go routine sending to a channel, the order in which the sends and receives occur is unspecified.

View

The View method is where we take the state of our Model and render it. Here we will be using the libraries **lipgloss** and **bubbles** that we installed previously.

[view.go](#)

```
package main

import (
    "strings"

    "github.com/charmbracelet/lipgloss"
)

var (
    appNameStyle =
lipgloss.NewStyle().Background(lipgloss.Color("99")).Padding(0, 1)

    faint =
lipgloss.NewStyle().Foreground(lipgloss.Color("255")).Faint(true)

    listEnumeratorStyle =
lipgloss.NewStyle().Foreground(lipgloss.Color("99")).MarginRight(1)
)
```

```

func (m model) View() string {
    s := appNameStyle.Render("NOTES APP") + "\n\n"

    if m.state == titleView {
        s += "Note title:\n\n"
        s += m.textinput.View() + "\n\n"
        s += faint.Render("enter - save • esc - discard")
    }

    if m.state == bodyView {
        s += "Note:\n\n"
        s += m.textarea.View() + "\n\n"
        s += faint.Render("ctrl+s - save • esc - discard")
    }

    if m.state == listView {
        for i, n := range m.notes {
            // render each note
        }
        s += faint.Render("n - new note • q - quit")
    }

    return s
}

```

Again, as we have other models like `textarea` and `textinput`, we call `View()` on them too to embed them into our top-level view. Also, because the `View()` method returns a simple string, it becomes easy to test this deterministic function.

Conclusion

I didn't include all the code, like SQLite storage and some utils, but you can find the [full program here](#) and even run it yourself, all you need is Go installed.

Building interactive TUIs in Go is genuinely enjoyable, thanks to powerful libraries like Bubble Tea.

To me the main strength of Bubble Tea is its simplicity with Elm Architecture, allowing developers to craft robust and responsive TUI applications, but also very modular.

The applications developed using Bubble Tea are usually very performant as well.

Let me know in the comments below if you built anything fun with Bubble Tea. I for example built this simple [ultrafocus](#) TUI to block distracting websites and boost productivity.

Cut Code Review Time & Bugs in Half (Sponsor)

Cut Code Review Time and Bugs in *Half* with CodeRabbit

```
src/App/App.jsx
... @@ -247,6 +247,7 @@ function App({ dexieCurrentUserInstance }) {
247 247   Like so: https://preview.app2.datamermid.org/123/index.html */
248 248   <Route exact path="/index.html" element={<Navigate to="/projects"
249 249   <Route path="/" element={<PageNotFound /> } />
250 +   <Route path="/" element={<PageNotFound /> } />

coderabbitai bot yesterday
Verification agent
Analysis chain
Duplicate Catch-All Route Detected
The file src/App/App.jsx currently defines two catch-all routes that render the <PageNotFound /> component:
• Line 249: <Route path="/" element={<PageNotFound /> } />
• Line 250: <Route path="/" element={<PageNotFound /> } />
Both routes serve the same purpose of catching undefined paths. To clean up the code and avoid potential confusion, please remove one of these redundant route definitions. For consistency, it's recommended to keep the <Route path="/" element={<PageNotFound /> } /> and remove the other.
Refactor suggestion:
<Route path="/" element={<PageNotFound /> } />
- <Route path="/" element={<PageNotFound /> } />
Committable suggestion
```

Code reviews are critical but time-consuming. [CodeRabbit](#) acts as your AI co-pilot, providing instant Code review comments and potential impacts of every pull request.

Beyond just flagging issues, CodeRabbit provides one-click fix suggestions and lets you define custom code quality rules using AST Grep patterns, catching subtle issues that traditional static analysis tools might miss.

CodeRabbit has so far reviewed more than 10 million PRs, installed on 1 million repositories, and used by 70 thousand Open-source projects. CodeRabbit is free for all open-source repo's!



16 Likes • 1 Restack

Previous

Discussion about this post

Comments Restacks



Write a comment...



Bastian Bergmann 1d



♥ Liked by Alex Pliutau

Alex!!! You legend!

♥ LIKE (1) 💬 REPLY

🔗 SHARE

1 reply by Alex Pliutau

1 more comment...